

LWC Task 1 – Inventory Dashboard

Task Completion & Debugging Report

Student	Ajher Mahmud Robin
Submission	30278_Ajher_Mahmud_Robin_LWC_TASK1.zip
Components	productItem (child), inventoryDashboard (parent)
Platform	Salesforce Lightning Web Components (LWC)

1. Task Overview

The assignment required building a parent-child Lightning Web Component pair demonstrating core LWC data-flow concepts: public property binding (@api), computed getters, custom event communication from child to parent, debounced search input, and immutable array state updates.

Component responsibilities:

- **productItem** (child) – displays a single product's data and notifies the parent when a unit is sold, without mutating any data itself.
- **inventoryDashboard** (parent) – owns the product array as the single source of truth, renders a filtered, searchable list of children, and recomputes total inventory valuation reactively.

2. Requirements Checklist – Final Status

#	Requirement	Status
A	Child: expose product data via @api properties, correctly mapped camelCase → kebab-case	✓ Done
B	Child: uppercase product name via a JS getter (no template string methods)	✓ Done
C	Child: dispatch a stockchange CustomEvent with the product id as payload; no internal mutation	✓ Done
A	Parent: hardcoded source-of-truth array of 3+ products (Id, Name, Price, Stock Count)	✓ Done
B	Parent: 300ms debounced search filtering the on-screen list	✓ Done
C	Parent: intercept stockchange, update stock count immutably (spread/map, no push/mutation)	✓ Done
D	Parent: getter computing total inventory valuation (Sum of Price × Stock Count)	✓ Done

3. Development & Debugging History

Development proceeded across several review passes. The table below summarizes the issues identified at each stage and how they were resolved before final submission.

Stage	Issue Found	Resolution
Initial draft	productItem still contained unrelated leftover properties (recordId, mode, showIcon) from reference demo files; no @api mapping to product data existed yet.	Replaced with product-specific @api properties (itemId, name, price, stockCount).
Initial draft	inventoryDashboard.js carried over unrelated lifecycle demo code (constructor, connectedCallback, errorCallback, handleCorrectArrayPush, etc.) with no connection to the actual task.	Removed; replaced with task-specific state and handlers.
Initial draft	Debounce timer fired but only logged to console; never filtered the array. Template did not render child components or a search input at all.	Introduced searchTerm + filteredItems getter driven by the debounced value; wired for:each over c-product-item in the template.
Mid revision	Template had a naming collision: for:item reused the array's own name, shadowing it inside the loop.	Renamed loop variable to a distinct identifier (item).
Pre-final review	inventoryDashboard.html contained unbalanced/broken markup: an empty leftover for:each block, a c-product-item tag placed outside any loop, a stray "</c-product-item>div>" text fragment, an orphaned , and extra unmatched closing </template> tags.	Rebuilt the template with balanced tags and the child correctly nested inside the for:each loop.
Pre-final review	Case-sensitivity mismatch: array field declared as myitemList, but referenced as myItemlList throughout getters, handlers, and template bindings.	Standardized every reference to the actual field name, myitemList.
Pre-final review	productItem.js declared @api id, which collides with the native id property every DOM/LWC element already inherits from HTMLElement.	Renamed the property to itemId to avoid the platform reserved-word conflict.

Stage	Issue Found	Resolution
Pre-final review	Getter/event handler in productItem.js referenced this.productName / this.productId, which no longer matched the declared @api property names, causing the uppercase getter to always return an empty string and the event payload to be undefined.	Updated getter and event handler to reference the correct property names (name, itemId).
Pre-final review	Getter name mismatch: JS defined get sumTotal(), while the template looked for {totalValuation}.	Aligned template binding to {sumTotal} to match the JS getter.
Final submission	All four files verified against the corrected reference implementation.	Confirmed identical logic (only harmless comment differences); no functional issues remaining.

4. Key Implementation Notes

Data flow (parent → child → parent)

inventoryDashboard owns myitemList as the single source of truth. Each visible product is rendered by looping over a derived filteredItems getter, passing individual fields down to c-product-item via @api attribute binding. When a unit is sold, the child never touches stock data directly – it dispatches a stockchange CustomEvent carrying only the product's id in event.detail. The parent listens via onstockchange, locates the matching item, and reassigns myitemList using .map() with object spread, ensuring a new array/object reference so LWC's reactivity re-renders correctly.

Debounced search

handleSearchInput clears any pending timeout on every keystroke and starts a fresh 300ms timer. Only after typing pauses does searchTerm update, which in turn causes the filteredItems getter to recompute and the template to re-render – avoiding re-filtering on every keystroke.

Immutability rule

Per the task's core constraint, all state updates use non-mutating patterns (array spread / .map() producing new objects) rather than .push() or in-place property assignment on existing array elements, since LWC's change detection is reference-based.

Reserved word avoided

The child's identifier property is named itemId rather than id, since id is already a native property inherited from HTMLElement on every LWC component and would otherwise conflict with the platform.

5. Final Verification

The submitted archive (30278_Ajher_Mahmud_Robin_LWC_TASK1.zip) was compared file-by-file against the fully corrected reference implementation. All four core files (productItem.js, productItem.html, inventoryDashboard.js, inventoryDashboard.html) matched the verified-correct logic, with only non-functional comment differences remaining in inventoryDashboard.js. Component metadata (isExposed, targets) was confirmed configured for lightning__AppPage, lightning__RecordPage, and lightning__HomePage, making the component deployable and viewable via Lightning App Builder in a scratch org.

Overall Verdict: READY FOR SUBMISSION

All eight task requirements (Child A/B/C, Parent A/B/C/D) are implemented correctly, with no compile-breaking markup, no naming mismatches, and no reserved-word conflicts remaining.

6. Line-by-Line Code Walkthrough

The final, submitted version of all four core files is explained below, line by line.

productItem.js

Line	Code	Explanation
1	<code>import { LightningElement, api } from 'lwc';</code>	Imports the base LWC class and the @api decorator used to expose properties/methods to the parent.
3	<code>export default class ProductItem extends LightningElement {</code>	Declares the component class, inheriting standard lifecycle and DOM behavior from LightningElement.
4–7	<code>@api itemId; / @api name; / @api price; / @api stockCount;</code>	Four public properties the parent binds data into via kebab-case attributes (item-id, name, price, stock-count). itemId is used instead of id because id is a reserved property already inherited from HTMLElement.
10–12	<code>get upperProductName() { return this.name ? this.name.toUpperCase() : ''; }</code>	A getter (Task B) that transforms the product name to uppercase entirely in JavaScript. Returns an empty string safely if name hasn't been passed yet, avoiding a runtime error on undefined.
15–22	<code>handleSellClick() { ... this.dispatchEvent(sellEvent); }</code>	Task C's event handler, triggered by the Sell button's onclick. It builds a CustomEvent named stockchange carrying only the product's itemId inside detail, then dispatches it upward. The child never modifies stockCount itself – it only reports which product was sold.

productItem.html

Line	Code	Explanation
2	<code><lightning-card title="Product" i con-name="standard:product_item"></code>	Wraps the component in a standard SLDS-styled card for consistent presentation.
4	<code><p>Name: {upperProductNam e}</p></code>	Binds directly to the JS getter (no parentheses) – displays the already-uppercased name.
5–6	<code><p>Price: ... {price}</p> / <p>Stock Count: ... {stockCount}</p></code>	Directly render the raw @api values passed down from the parent, no transformation needed.
8–13	<code><lightning-button label="Sell 1 Unit" ... onclick={handleSellClick} ...></code>	Renders the button named in the spec; clicking it calls handleSellClick, which dispatches the stockchange event (defined in the .js file).

inventoryDashboard.js

Line	Code	Explanation
1	<pre>import { LightningElement } from 'lwc';</pre>	No @api needed here since this is the top-level parent component; it doesn't receive props from anything above it.
5–9	<pre>myitemList = [{Id:1,...}, {Id:2,...}, {Id:3,...}];</pre>	Task A: the hardcoded source-of-truth array. Each object carries Id, Name, Price, and Stock_Count. Being a plain class field, reassigning it (not mutating it) triggers reactivity.
12–13	<pre>searchTerm = ''; / delayTimeout;</pre>	searchTerm holds the debounced (settled) search value the filter actually uses. delayTimeout stores the pending setTimeout id so it can be cancelled on each keystroke.
16–29	<pre>handleSearchInput(event) { ... window.setTimeout(() => { this.searchTerm = value; ... }, 300); }</pre>	Task B: on every keystroke, any pending timer is cleared (clearTimeout) and a new 300ms timer is started. Only once typing pauses for a full 300ms does searchTerm actually update, which is what the filteredItems getter reacts to.
31–39	<pre>get filteredItems() { if (!this.searchTerm) return this.myitemList;filter(...) }</pre>	Derives the list actually shown on screen. With no search term, the full inventory is shown; otherwise it returns only items whose Name includes the (lowercased) search term.
41–50	<pre>handleStockChange(event) { const targetId = event.detail.productId; this.myitemList = this.myitemList.map((item) => { if (item.Id === targetId) return {...item, Stock_Count: item.Stock_Count - 1}; return item; }); }</pre>	Task C: reads the sold product's id from the child's event payload, then rebuilds the entire array with .map(). For the matching item, a brand-new object is returned via spread with Stock_Count decremented; every other item is passed through unchanged. Reassigning myitemList (rather than mutating an element in place) is what makes this immutable and reactive.
52–57	<pre>get sumTotal() { return this.myitemList.reduce((sum, item) => sum + item.Price * item.Stock_Count, 0); }</pre>	Task D: sums Price × Stock_Count across the entire inventory (myitemList, not the filtered view, matching the spec's 'entire inventory' wording). Recalculates automatically whenever myitemList changes.

inventoryDashboard.html

Line	Code	Explanation
6–11	<code><lightning-input type="search" label="Search Products" onchange={handleSearchInput} ...></code>	The search box from Task B. Every onchange fires handleSearchInput, which restarts the 300ms debounce timer defined in the .js file.
13	<code><p>Showing {filteredItems.length} of {myitemList.length} items</p></code>	A small usability touch confirming the filter is working – shows how many items match the current search versus the full inventory size.
16	<code><template for:each={filteredItems} for:item="item"></code>	Iterates over the filtered (debounced-search) list rather than the raw array, so the visible cards update once the user stops typing.
17–25	<code><c-product-item key={item.Id} item-id={item.Id} name={item.Name} price={item.Price} stock-count={item.Stock_Count} onstockchange={handleStockChange} ...></code>	Renders one child per item. Each @api property on the child is fed via its kebab-case attribute (item-id, stock-count). onstockchange wires the child's dispatched CustomEvent back to the parent's handleStockChange handler. key={item.Id} satisfies LWC's requirement for a unique key inside for:each.
29–31	<code><div class="slds-box ..."><p>Total Inventory Valuation: \${sumTotal}</p></div></code>	Task D's summary panel, bound directly to the sumTotal getter, updating live whenever a sale changes myitemList or a search changes what's filtered (the total intentionally still reflects the whole inventory, not just the filtered subset).